

Experimento 2 – Registrador de deslocamento, usado como contador binário de 8 bits – parte 2

Neste projeto, será utilizado um CI registrador de deslocamento, para fazer com que os LEDs contem em binário.

O CI 74HC595 é um registrador de deslocamento de 8 bits, com entrada serial, saída serial ou paralela, e *latches* (travas) de saída. Isso significa que se pode enviar dados para o registrador de deslocamento em série, e enviar dados para fora em paralelo, como será utilizado neste projeto.

Os dados entram quando o LATCH está definido como LOW, e saem quando o LATCH está definido como HIGH. Assim, os dados são transferidos ao registrador de deslocamento, um bit de cada vez, e então enviados para fora oito bits ao mesmo tempo. Cada bit é deslocado conforme o bit seguinte entra. Se um nono bit entrar antes que o latch seja definido como HIGH, o primeiro bit será deslocado para o fim da fila, sendo perdido.

Registradores de deslocamento são geralmente utilizados na conversão de dados de formato serial para paralelo. Nesse caso, os dados emitidos são 0s e 1s (ou 0 V e 3,3 V/5V), por isso se pode utilizá-los para acender e apagar um conjunto de oito LEDs.

O registrador de deslocamento para este projeto requer apenas três portas digitais do Arduino. As saídas do Arduino e as entradas do 74HC595 podem ser vistas na tabela 1.

Tabela 1 – Pinos utilizados

Pino do Arduino	Pino do 74HC595	Ref.	Descrição
8	12	latch	Entrada do clock do registrador de armazenamento
11	14	dados	Entrada dos dados seriais
12	11	clock	Entrada do clock do registrador de deslocamento

Para facilitar a leitura serão consideradas as seguintes referências: O pino 11 será o pino do clock, o pino 14 o pino dos dados e o pino 12 como o pino do latch. Imagine o latch como um portão que permite aos dados saírem do 74HC595. Quando o portão está abaixado (LOW), os dados no 74HC595 não podem escapar, mas outros dados podem entrar. Quando o portão está levantado (HIGH), dados não podem mais entrar, mas os dados no registrador de deslocamento são liberados para os oito pinos (QA-QH, ou de Q0 a Q7), dependendo de seu datasheet; consulte a figura 1. O clock é formado por pulsos de 0s e 1s, e o pino dos dados é por onde os dados são enviados do Arduino para o [Digite texto]

74HC595.

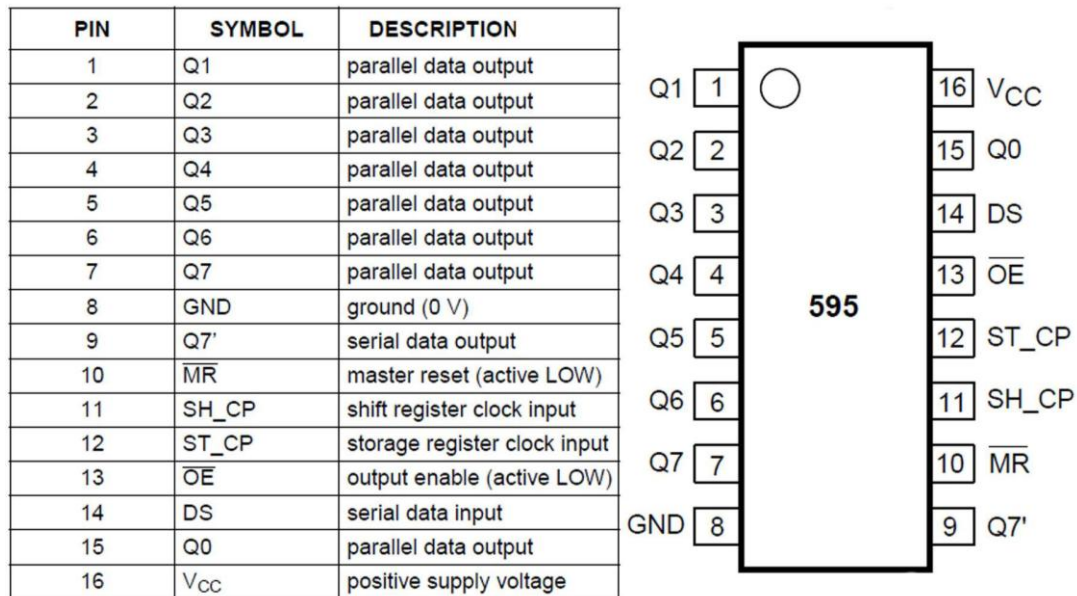


Figura 1 – Diagrama e descrição dos pinos do um chip 595.

Funcionamento

Para utilizar o registrador de deslocamento, o pino do latch e o pino do clock devem estar definidos como LOW. O pino do latch permanecerá LOW até que todos os oito bits tenham sido definidos. Isso permite que os dados entrem no registrador de armazenamento (um local no CI para armazenamento de 1 ou 0). Na sequência, é apresentado um sinal HIGH ou LOW no pino de dados, e se define o pino do clock como HIGH. Ao fazê-lo são armazenados no registrador de armazenamento os dados apresentados no pino de dados. Feito isso, é definido novamente o clock como LOW, e apresentado no pino de dados o próximo bit de dados. Fazendo isso oito vezes, terá sido enviado um número completo de 8 bits ao 74HC595. O pino do latch agora será levantado o que transferirá os dados do registrador de armazenamento para o registrador de deslocamento, fazendo sua saída de Q0 (pino 15) a Q7 (pinos de 1 a 7). Essa sequência de eventos está descrita na tabela 2.

Assim, têm-se oito LEDs conectados às oito saídas do registrador. O latch é definido como LOW, para habilitar a entrada de dados. Dados são enviados ao pino de dados, um bit por vez, e o pino do clock é definido como HIGH para armazenar os dados. Depois, esse pino é definido como LOW novamente, para aguardar o próximo bit. Depois da entrada de todos os oito bits, o latch é definido como HIGH, impedindo a entrada de novos dados e definindo o estado dos oito pinos de saída como alto (3,3 V) ou baixo (0 V), dependendo do estado do registrador.

[Digite texto]

Tabela 2 – Sequência de eventos

Pino	Estado	Descrição
Latch	LOW	Latch abaixado, para permitir que os dados entrem
Dados	HIGH	Primeiro bit de dado (1)
Clock	HIGH	Clock vai para HIGH. Dados armazenados
Clock	LOW	Pronto para o próximo bit. Impede novos dados
Dados	HIGH	2° bit de dado (1)
Clock	HIGH	2° bit armazenado
...	...	...
Dados	LOW	8° bit de dado (0)
Clock	HIGH	8° bit armazenado
Clock	LOW	Impeça que novos dados sejam armazenados
Latch	HIGH	Envie os 8 bits em paralelo

A sequência dos eventos descritos na seção Funcionamento acima está implementada na função `shiftOut` que está na categoria *advanced i/o*, dentro da documentação de referência da linguagem do Arduino. Maiores informações podem ser obtidas no link:

<https://docs.arduino.cc/language-reference/pt/en/functions/advanced-io/shiftOut/>

Com isso o esforço de programação fica reduzido, não sendo necessário realizar a programação descrita na seção funcionamento.

A função `shiftOut`

A função `shiftOut` do Arduino está descrita na categoria *Advanced i/o* e ela é adequada para se lidar com o CI 74595.

Essa função transfere um byte, bit a bit, ou seja, um bit por vez. Pode-se iniciar como o bit mais significativo (MSB) ou menos significativo (LSB). Cada bit é escrito em sequência em um pino de dados (pino 14 do 74595), em seguida o pino de clock é levado a LOW e depois a HIGH, isso indica que o bit está disponível.

No caso do 74595, que é sincronizado pela borda de subida do sinal de clock, antes de chamar a função `shiftOut` você deverá colocar o pino de clock em LOW, chamar a função, e em seguida setá-lo para HIGH.

[Digite texto]

Sintaxe:

`shiftOut(dado, clock, bitOrder, valor);`

Parâmetros:

dado: o pino do Arduino no qual transferir cada bit (int), que estará conectado ao pino 14 do 74595.

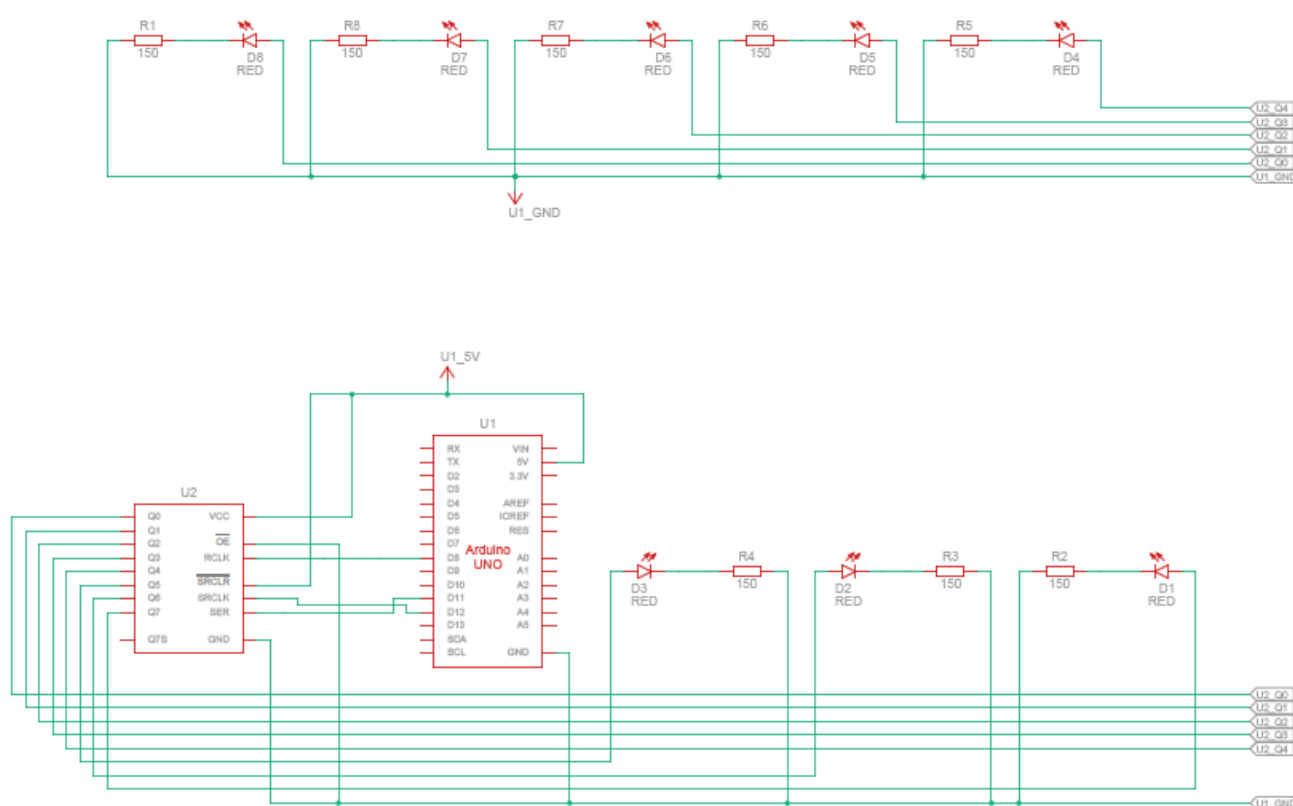
clock: o pino do Arduino a ser pulsado uma vez que o pino data estiver com o bit a ser transferido (int), que estará conectado ao pino 12 do 74595.

bitOrder: em que ordem receber os bits; pode ser `MSBFIRST` ou `LSBFIRST`. Respectivamente, primeiro o bit mais significativo (Most Significant Bit First), ou o primeiro o bit menos significativo (Least Significant Bit First)

valor: o valor a ser transferido. (byte).

Montagem do Circuito

Aproveite os materiais utilizados na parte 1 deste experimento e realize a montagem conforme o diagrama esquemático da figura 2. Poderá se valer também da figura 3 com o diagrama de componentes e ligações feitos no Tinkercad.



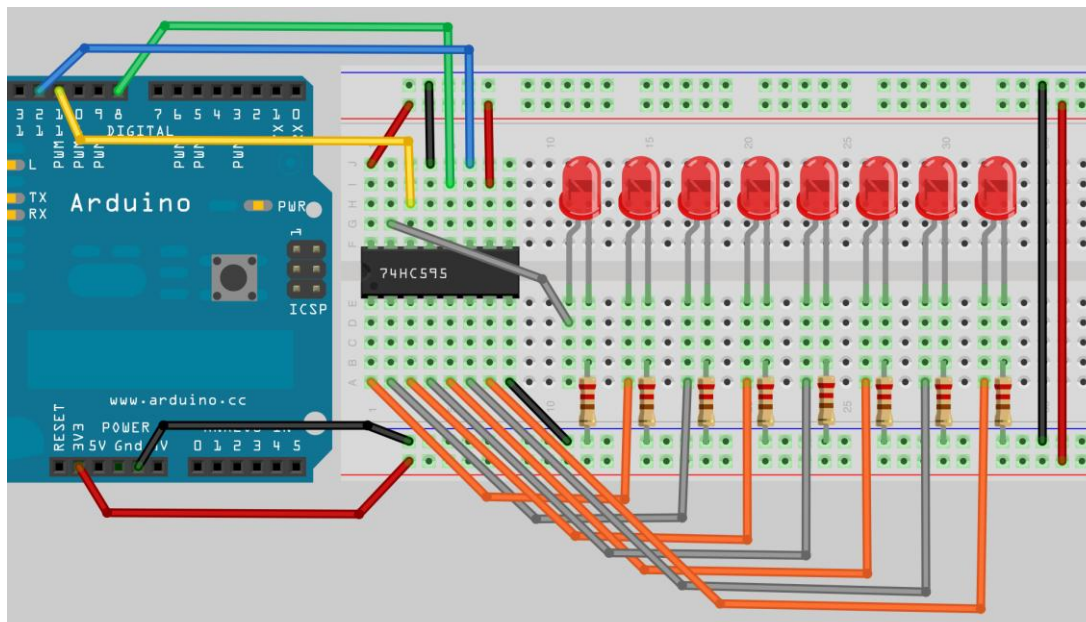


Figura 3 - Diagrama de componentes e ligações

Material necessário:

- 1) Arduino UNO R3.
- 2) Protoboard.
- 3) Fios de ligação;
- 4) 1 CI registrador de deslocamento 74HC595;
- 5) 8 resistores de 150 Ω ;
- 6) 8 LEDs de 5 mm.

Código:

No seu código você deve definir três portas digitais como saída, vide tabela, como referência.

A função loop principal executa um loop for contando de 0 a 255. Em cada iteração do loop, o pino latch é definido como LOW, para permitir a entrada de dados. Na sequência, é chamada a função shiftOut do Arduino, passando os parâmetros descritos na sintaxe dela. Depois, o pino latch é definido como HIGH, impedindo a entrada de mais dados e definindo as saídas dos oito pinos. Por fim, utilize a função delay, com um valor a ser configurado por você, antes que se inicie a próxima iteração do loop.

Relatório: em seu relatório, além dos tópicos a serem abordados, analise as vantagens e desvantagens entre se utilizar diretamente os pinos digitais para fazer o contador binário de 8 bits e a opção de se utilizar o registrador de deslocamento.

[Digite texto]